# Üzembe helyezés

VPS-től az éles HTTPS-ig · 2026. szeptember

Ez a dokumentum végigvezet a nulláról az éles, HTTPS-en elérhető rendszerig, és leírja a frissítés, mentés és hibakeresés menetét.

## 1. Miből áll a futó rendszer

Három konténer:

| Konténer | Szerep | Publikált port |
|---|---|---|
| `web` | Caddy: a beépített frontendet szolgálja ki, proxyzza az API-t és a WebSocketet (`/api/*`), kezeli a TLS-t | 80, 443 |
| `api` | FastAPI + MediaPipe: gesztusfelismerés | — |
| `db` | PostgreSQL 16: hozzájárulás és mérési adatok | — |

Csak a `web` érhető el kívülről. Az API és az adatbázis kizárólag a belső Docker hálózaton, ami azt jelenti, hogy az adatbázis nincs kitéve az internetnek.

## 2. Előfeltételek

- Egy VPS vagy szerver, legalább **2 CPU mag és 4 GB RAM** (a MediaPipe CPU-n fut; ennél kevesebbel több párhuzamos felhasználónál akadozik).
- Docker Engine és a Compose plugin.
- Egy domain vagy aldomain, amelynek **A rekordja már a szerver IP-címére mutat**. A tanúsítványigénylés csak akkor sikerül, ha a DNS már él.
- A 80-as és 443-as port szabad és kívülről elérhető (tűzfal!).

A kamera használatához a böngésző **biztonságos környezetet** követel: HTTPS vagy localhost. Éles használatnál tehát a TLS nem opcionális, e nélkül a felhasználó kamerája el sem indul.

## 3. Telepítés

```
# 1. Kód
git clone <repo> gesture-vision && cd gesture-vision

# 2. Konfiguráció
cp .env.example .env
```

A `.env`-ben legalább ezeket állítsd be:

```
POSTGRES_PASSWORD=<hosszú, véletlen jelszó>
DATABASE_URL=postgresql+psycopg2://gesture:<ugyanaz a jelszó>@db:5432/gesture_db
SITE_ADDRESS=gesztus.pelda.hu
ACME_EMAIL=te@pelda.hu
ALLOWED_ORIGINS=https://gesztus.pelda.hu
```

Jelszógenerálás:

```
python3 -c "import secrets; print(secrets.token_urlsafe(24))"
```

Indítás:

```
./tools/deploy.sh
```

A szkript ellenőrzi az előfeltételeket, menti az adatbázist (ha már fut), buildel, elindít, megvárja az egészséges állapotot, majd lefuttatja az ellenőrzést. Hiba esetén megáll és kiírja a naplót — így nem jelent sikert egy félig működő telepítés. Kézzel ugyanez:

```
docker compose -f compose.prod.yaml up --build -d
```

Az első indulás néhány percet vesz igénybe: a MediaPipe telepítése és a frontend buildje a leghosszabb lépés. A Caddy a háttérben megszerzi a tanúsítványt.

## 4. Ellenőrzés

```
python3 tools/smoke_test.py https://gesztus.pelda.hu
```

Amit végigmér: API állapot, frontend kiszolgálás, munkamenet kiadása, hozzájárulás rögzítése, token nélküli kérés elutasítása, WebSocket kézfogás, képkocka körbeérési ideje, túl nagy képkocka elutasítása, biztonsági fejlécek. Hiba esetén nem nulla kilépési kóddal tér vissza, így telepítő szkriptbe köthető.

Konténerállapot és naplók:

```
docker compose -f compose.prod.yaml ps
docker compose -f compose.prod.yaml logs -f api
docker compose -f compose.prod.yaml logs web | grep -i "certificate\|error"
```

## 5. Frissítés

```
./tools/deploy.sh --pull
```

Ez sorrendben: mentés → `git pull` → build → indítás → ellenőrzés. Kézzel:

```
./tools/backup_db.sh
git pull
docker compose -f compose.prod.yaml up --build -d
python3 tools/smoke_test.py https://gesztus.pelda.hu
```

Az adatbázis sémája induláskor automatikusan létrejön, illetve kiegészül az új táblákkal. Meglévő tábla szerkezetét nem módosítja — ha egy oszlop típusa változna, azt kézzel kell migrálni.

## 6. Mentés

A mérési adatok nem gyűjthetők újra, ezért a mentés nem opcionális.

```
./tools/backup_db.sh                 # ./backups mappába
KEEP_DAYS=90 ./tools/backup_db.sh /mnt/backup
```

Napi automatikus mentés:

```
crontab -e
# minden nap hajnali 3-kor
0 3 * * * cd /opt/gesture-vision && ./tools/backup_db.sh >> /var/log/gesture-backup.log 2>&1
```

Visszaállítás:

```
gunzip -c backups/gesture_db-20260915-030000.sql.gz \
```

```
  | docker compose -f compose.prod.yaml exec -T db psql -U gesture -d gesture_db
```

A kutatási adatok exportja elemzéshez:

```
docker compose -f compose.prod.yaml exec api python3 /srv/meresek/export_data.py --out /srv/export
docker compose -f compose.prod.yaml cp api:/srv/export ./meresek/data
python3 meresek/analyze.py meresek/data/<időbélyeg>
```

## 7. Terhelés és méretezés

Az egyidejű munkamenetek számát a `MAX_CONCURRENT_SESSIONS` korlátozza (alapérték 25). Minden munkamenet saját MediaPipe példányt tart, ami memóriát és CPU-t köt le, ezért ez a korlát véd a túlterheléstől: a limit felett a WebSocket `4503` kóddal elutasít, a felhasználó pedig „a szerver megtelt" üzenetet lát a néma hiba helyett.

Hangolási pontok a `.env`-ben:

| Változó | Hatás |
|---|---|
| `MAX_CONCURRENT_SESSIONS` | egyidejű felhasználók felső korlátja |
| `MAX_FRAMES_PER_SECOND` | kliensenkénti képkocka-korlát (CPU-védelem) |
| `API_CPU_LIMIT`, `API_MEMORY_LIMIT` | konténer erőforrás-korlátai |
| `REST_RATE_LIMIT_PER_MINUTE` | REST végpontok kérésszám-korlátja |

Ha több felhasználót kell kiszolgálni, mint amennyit egy gép bír: az API állapotmentes a munkameneten kívül, de **egy munkamenetnek ugyanahhoz a példányhoz kell csatlakoznia** (a detektor a kapcsolathoz kötött). Több példány esetén tehát ragadós (sticky) terheléselosztás kell, a munkamenet-tokenre kulcsolva.

## 8. Hibakeresés

| Tünet | Valószínű ok |
|---|---|
| A kamera nem indul el | Nem HTTPS-en érted el az oldalt. A böngésző csak biztonságos környezetben ad kamerát. |
| A tanúsítvány nem jön létre | A DNS még nem mutat a szerverre, vagy a 80-as port zárva. A Caddy naplója megmondja. |
| `4401` a WebSocketen (`/api/ws`) | Lejárt vagy érvénytelen munkamenet-token. A kliens automatikusan újat kér; ha ismétlődik, nézd meg a `SESSION_TOKEN_TTL` értékét. |
| `4503` a WebSocketen | Elérted a `MAX_CONCURRENT_SESSIONS` korlátot. |
| Akadozó felismerés | Kevés a CPU. Nézd meg a `docker stats` kimenetét, csökkentsd a `MAX_FRAMES_PER_SECOND` értéket, vagy adj több magot. |
| `database: disabled` az állapotban | Hiányzó vagy hibás `DATABASE_URL`. A rendszer szándékosan fut tovább adatbázis nélkül, de ilyenkor nem gyűjt mérési adatot. |

## 9. Folyamatos ellenőrzés (CI)

A `.github/workflows/ci.yml` minden `main`-re küldött változtatásnál és minden pull requestnél lefuttatja:

- a frontend buildet (a figyelmeztetések hibának számítanak),
- a backend importálását és egy detektor-ellenőrzést,
- a **teljes production stack** felépítését és indítását, majd a `smoke_test.py`-t ellene.

Ez azért fontos, mert a stack így minden változtatásnál valódi Docker környezetben is bizonyítottan elindul — nem csak azon a gépen, ahol a fejlesztés zajlik. Hiba esetén a konténernaplók a futás kimenetében megjelennek.

## 10. Adatvédelmi megjegyzés az éles üzemhez

A rendszer nyers képkockát sem lemezre, sem adatbázisba nem ír; a kamerakép kizárólag a memóriában, a felismerés idejére létezik. Az adatbázis metaadatot tárol: gesztus neve, időbélyeg, megbízhatóság, késleltetés, munkamenet-azonosító, valamint a hozzájárulás ténye verzióval és időponttal.

Ha a rendszert kutatási adatgyűjtésre használod, a hozzájárulási szöveg a `consentDialog` fordítási kulcsokban szerkeszthető, és a `CONSENT_VERSION` értékét érdemes növelni, amikor a szöveg érdemben változik — így az adatbázisban látszik, ki melyik verzióra adott hozzájárulást.